

Plain-to-MeTTa and Plain-to-Rholang: A SpecAtom IR Proposal

ZeroBot / ProtoCosmoBot working note for Ben Goertzel

2026-06-29

Abstract

This note reviews a Telegram discussion among Ben Goertzel, ProtoCosmoBot, and ProtoMegaTron about building a toolchain from the *****plain** software specification language to MeTTa, initially PeTTa, and later Rholang and MeTTa-IL. The central recommendation is to avoid a direct natural-language to code generator. Instead, parse the structured parts of Plain into a small, typed, Atomspace-like logical intermediate representation, called here *SpecAtom IR*. This IR should be close enough to MeTTa that MeTTa/PeTTa lowering is natural, while preserving enough type and proof structure to support later OSLF-style Curry-Howard correspondences and PLN-based spec analysis. The MVP should use only shallow natural-language processing but serious type discipline.

1 Context and motivation

Ben pointed to the Plain documentation and GitHub organization and asked about the viability of a tool that maps Plain software specifications into MeTTa, using PeTTa as the first concrete MeTTa dialect, and then into Rholang. A later extension to MeTTa-IL was also proposed.

Plain, as described in its documentation, is a textual specification language for spec-driven development. Its structure includes sections such as:

- definitions;
- implementation requirements;
- test requirements;
- functional specifications;
- acceptance tests nested under functional specifications.

The key observation is that Plain is not just free-form prose. It already has a sectional structure that separates concepts, implementation choices, behavior, and tests. That structure is sufficient for a useful first compiler even if the English content of each bullet is only minimally interpreted.

The discussion converged on the view that the project is viable, but should be designed as a staged semantic compiler rather than a direct prose-to-code translator:

Plain -> PlainAST -> SpecAtom IR -> MeTTa/PeTTa -> Rholang / MeTTa-IL

2 Why not compile Plain directly to MeTTa or Rholang?

A direct compiler from Plain text to MeTTa or Rholang would be tempting, but it would prematurely mix several distinct semantic roles:

- concepts and types;
- definitional assertions;
- behavioral obligations;

- implementation constraints;
- test oracles;
- constructive witnesses or program fragments;
- process and concurrency constraints.

If these roles are collapsed, the resulting tool becomes mostly a code generator with opaque prompt engineering. The more interesting possibility is to make the specification inspectable, queryable, transformable, and analyzable before code is emitted.

This suggests a logical IR. The IR can preserve the role of each Plain clause, while still being easy to lower to executable or semi-executable targets.

3 SpecAtom IR: an Atomspace-like logical mirror

Ben suggested that the IR may best be an Atomspace-based logical representation. There are two major reasons.

3.1 A logical mirror of MeTTa

If the IR is designed as a logical mirror of MeTTa, then the mapping to MeTTa code is not an ad hoc backend. It becomes a lowering from typed logical objects to executable or rewrite-oriented MeTTa forms. This is especially appealing if the type relationships are arranged to support Curry-Howard-style and Meredith/Stay OSLF-style correspondences.

The guiding idea is that specifications, types, proofs, programs, tests, and processes should not be unrelated syntactic classes. They should be related by explicit correspondences. For example:

Spec role	Logical/programmatic role
Concept	Type or sort
Functional requirement	Proposition or obligation
Implementation fragment	Constructive witness or program term
Acceptance test	Refutation attempt, observation predicate, or validation procedure
Process requirement	Behavioral protocol or transition-system constraint
Refinement relation	Proof obligation or decomposition relation

This table is not yet a formal OSLF mapping, but it indicates the kind of type separation needed from the start.

3.2 Future PLN-based spec analysis

An Atomspace-like IR also opens the door to PLN-based analysis. Requirements are often incomplete, uncertain, redundant, or inconsistent. If they are represented as typed logical objects rather than opaque text, then later tools can ask:

- Are two requirements contradictory?
- Is a concept underspecified?
- Does an acceptance test actually test its parent functional requirement?
- Is an implementation requirement inconsistent with a process requirement?
- Are there missing tests for high-risk obligations?
- Which Plain clauses support or weaken a proposed MeTTa or Rholang artifact?

For an MVP, such PLN analysis need not be implemented. The important point is to choose an IR that does not block it.

4 A small typed schema for the MVP

The main danger of an Atomspace-like IR is overgeneralization. A full Atomspace ontology could consume the project before a working prototype exists. The MVP should therefore define a deliberately small schema, perhaps called `SpecAtom`. The schema should be narrow enough for deterministic parsing from Plain sections but typed enough to support later logical correspondences.

A possible initial vocabulary is:

```
(PlainFile file-id path)
(Section section-id file-id SectionKind)
(PlainItem item-id section-id ordinal raw-text)

(Concept concept-id name)
(Defines item-id concept-id raw-text)
(Attribute concept-id attribute-id Requiredness ValueKind raw-text)

(ImplementationReq req-id item-id raw-text)
(TestReq req-id item-id raw-text)
(FunctionalReq req-id item-id raw-text)
(AcceptanceTest test-id parent-functional-req-id raw-text)

(Refines child-req-id parent-req-id)
(DependsOn req-id other-req-id)
(Constrains constraint-id target-id raw-text)

(Obligation req-id proposition-id)
(Witness witness-id obligation-id target-language artifact-ref)
(TestOracle test-id predicate-id)
(ProcessConstraint constraint-id protocol-id)
```

This is intentionally modest. The raw English remains present. The compiler does not pretend to understand all English. However, each artifact receives a logical role and a place in the type structure.

Principle 1 (Role preservation). *Every generated artifact should know whether it is a type or concept, a proposition/spec obligation, a constructive witness/program fragment, a test/refutation condition, or a process/interaction constraint.*

Principle 2 (Raw-text provenance). *Every logical object generated from Plain should retain a pointer to the source file, section, bullet, and raw text span that produced it.*

Principle 3 (Shallow NLP first). *The MVP should avoid full English-to-logic parsing. It should parse Plain's explicit structure and concept references, then preserve the remaining prose as raw text attached to typed logical objects.*

5 Mapping Plain to SpecAtom IR

The Plain-to-IR compiler can be mostly deterministic at first.

5.1 Definitions

Plain definitions introduce concepts and possibly attributes. For example:

```
***definitions***
- :Task: describes an activity that needs to be done by :User:. :Task:
  has the following attributes:
  - Name - a short description of :Task:. This is a required attribute.
  - Due Date - optional date by which :User: is supposed to complete :Task:.
```

A shallow compiler can emit:

```
(Concept :Task "Task")
(Concept :User "User")
(Defines item-17 :Task "describes an activity ...")
(Attribute :Task :Name Required Text "a short description ...")
(Attribute :Task :DueDate Optional Date "optional date ...")
```

The exact attribute typing can begin heuristic-based and become more precise later.

5.2 Implementation requirements

Plain implementation requirements describe how the software should be built. They should not be confused with behavioral requirements. For example:

```
- :Implementation: should be in Python.
```

This may become:

```
(ImplementationReq impl-1 item-21 ":Implementation: should be in Python.")
(Constrains impl-constraint-1 :Implementation "target-language=Python")
```

The first MVP can store most of this as typed raw text. Later versions can recognize standard implementation templates.

5.3 Functional specifications

Functional specs describe behavior. A bullet such as:

```
- :User: should be able to add :Task:. Only valid :Task: items can be added.
```

can become:

```
(FunctionalReq req-33 item-33 raw-text-33)
(Obligation req-33 prop-33)
(References req-33 :User)
(References req-33 :Task)
```

A later semantic pass may infer action structure:

```
(ActionSchema action-33 :User add :Task)
(Precondition action-33 valid-task)
```

But the MVP does not need to rely on that inference.

5.4 Acceptance tests

Acceptance tests should be attached to their parent functional requirement and represented as test oracles or observation predicates:

```
(AcceptanceTest test-34 req-33 raw-test-text)
(TestOracle test-34 predicate-34)
(Tests test-34 req-33)
```

This is important because it lets later analysis ask whether a test actually covers its parent obligation.

6 Mapping SpecAtom IR to MeTTa/PeTTa

The first backend should be MeTTa/PeTTa because the IR is naturally atom-shaped. There are two levels of output.

6.1 Reified specification atoms

The simplest output is a faithful MeTTa representation of the IR:

```
(: Task Concept)
(Defines item-17 Task "describes an activity ...")
(Attribute Task Name Required Text)
(FunctionalReq req-33 item-33)
(References req-33 User)
(References req-33 Task)
(AcceptanceTest test-34 req-33)
```

This is already useful. It lets MeTTa queries inspect the specification, trace requirements, and drive later transformations.

6.2 Executable or semi-executable PeTTa skeletons

A second pass can lower selected functional requirements into executable skeletons. For example, if a functional requirement is recognized as an action, the compiler may generate PeTTa rules or stubs such as:

```
(: add-task (-> User Task TaskList TaskList))
(= (add-task $u $task $list)
   (if (valid-task $task)
       (append-task $task $list)
       (error InvalidTask)))
```

For the MVP, these skeletons can be conservative. If the compiler is unsure, it should emit a typed obligation plus a TODO witness rather than hallucinating an implementation.

7 Mapping SpecAtom IR to Rholang

Rholang is a less direct target than MeTTa because it is process-oriented. The IR should only lower to Rholang when the Plain specification includes enough process-like structure:

- services or agents;
- messages or events;
- channels or resources;
- state transitions;
- concurrency or consensus constraints;
- protocols with observable traces.

The Rholang backend should therefore be a later stage, not the first backend. It can consume IR objects such as:

```
(Process :TaskService)
(Channel :AddTaskRequest)
(Channel :TaskAddedEvent)
(Transition add-task-transition pre-state message post-state)
(ProcessConstraint c-12 protocol-7)
```

and emit Rholang process templates. Requirements that are purely functional or local should remain in MeTTa/PeTTa or be represented as comments/obligations in Rholang until a process interpretation is supplied.

8 OSLF-style type relationships

Ben emphasized that even without full English-to-logic parsing, the type relationships should be correct so that OSLF-style correspondences work later. This suggests the following initial type hierarchy.

```
SpecObject
  SourceObject
    PlainFile
    Section
    PlainItem
  LogicalObject
    Concept
    Proposition
    Obligation
    Constraint
    Refinement
  ConstructiveObject
    Witness
    ProgramFragment
    MeTTaTerm
    RholangProcess
  ValidationObject
    AcceptanceTest
    TestOracle
```

ObservationPredicate
 ProcessObject
 Channel
 Message
 Protocol
 Transition

The key correspondence pattern is:

Type-theoretic role	SpecAtom role
Type	Concept or interface sort
Term	Program fragment, action, or process term
Proposition	Functional requirement or constraint
Proof/witness	Implementation artifact satisfying an obligation
Counterexample	Failed acceptance test, trace violation, or model check failure
Refinement	Decomposition of an obligation into smaller obligations

This is enough to guide the MVP without forcing a complete formal semantics.

9 PLN-based analysis enabled later

Once Plain specs are represented as SpecAtom IR, PLN-style reasoning can operate on the specification graph. Examples include:

- estimating whether a functional requirement is supported by acceptance tests;
- detecting probable contradictions among requirements;
- identifying concepts used without definitions;
- suggesting refinements for overly complex Plain functional specs;
- propagating confidence from tests to obligations and witnesses;
- comparing two implementations as alternative witnesses for the same obligation;
- asking whether a Rholang process preserves the obligations of the MeTTa specification.

The MVP does not need PLN. It needs to avoid destroying the information that PLN would require later.

10 Proposed MVP

A realistic MVP could be:

1. Parse a subset of Plain files: section headers, bullet hierarchy, concept notation, and acceptance-test nesting.
2. Emit a JSON or S-expression PlainAST for debugging.
3. Emit SpecAtom IR in MeTTa syntax.
4. Provide simple MeTTa queries: list concepts, list obligations, list tests for an obligation, list undefined concept references.

5. Generate a conservative PeTTa skeleton for one small example, such as a hello-world app or task-manager fragment.
6. Preserve all raw text and source locations.
7. Reject or mark as underspecified anything beyond the shallow subset.

The MVP should not attempt full English parsing, automatic correct code synthesis, or general Plain-to-Rholang generation.

11 Development plan

11.1 Phase 0: corpus and parser

Collect a few Plain examples from the public examples repository and hand-write one or two examples aimed at MeTTa/PeTTa. Implement a parser for Plain’s section structure and bullet hierarchy.

11.2 Phase 1: SpecAtom schema

Define the small SpecAtom vocabulary. Write schema validation rules so that each artifact has exactly one primary role and a source pointer.

11.3 Phase 2: MeTTa/PeTTa backend

Emit faithful reified specification atoms. Add simple query examples and a small PeTTa skeleton generator for recognized action patterns.

11.4 Phase 3: type correspondence checks

Add checks inspired by Curry-Howard/OSLF:

- every witness targets an obligation;
- every test targets an obligation;
- every obligation references defined concepts or explicitly external concepts;
- refinements preserve parent-child role consistency;
- process constraints are not lowered to Rholang unless process roles are present.

11.5 Phase 4: Rholang backend

Add a restricted process sublanguage and emit Rholang templates only from that sublanguage.

11.6 Phase 5: PLN analysis

Layer PLN-style uncertainty and support relations over the SpecAtom graph. Use this for contradiction detection, coverage estimates, and refinement recommendations.

12 Risks and mitigations

Risk 1 (Overbuilt ontology). *A full Atomspace ontology could delay a working compiler indefinitely.*

Mitigation: define a tiny SpecAtom subset and require end-to-end Plain-to-MeTTa output in the first prototype.

Risk 2 (False semantic precision). *The compiler may appear to understand English that it only stores as raw text.*

Mitigation: mark shallowly parsed clauses explicitly. Use confidence, source pointers, and TODO witnesses rather than pretending full semantic interpretation.

Risk 3 (Rholang mismatch). *Many Plain specs may not contain enough process structure for meaningful Rholang output.*

Mitigation: require explicit process annotations before Rholang lowering.

Risk 4 (Loss of OSLF compatibility). *If early shortcuts collapse obligations, witnesses, tests, and processes, later OSLF-style mappings become awkward.*

Mitigation: enforce role preservation from the start, even if the English interpretation is shallow.

13 Conclusion

The proposed Plain-to-MeTTa and Plain-to-Rholang project is viable and potentially valuable, but its core should be a typed Atomspace-like IR rather than direct code generation. The simplest useful prototype should parse Plain’s explicit structure, preserve raw text and provenance, generate typed SpecAtom objects, and lower those objects to MeTTa/PeTTa. This is sufficient to make Plain specs inspectable and queryable while preserving a path toward OSLF-style type correspondences and PLN-based analysis.

The guiding maxim is: keep the first compiler shallow in natural-language understanding but deep in type discipline.